Python Programs – All 36 Questions

I'll follow your question bank in the same order and use simple, exam-friendly Python programs. Each answer will include the algorithm, program, and sample output wherever applicable.

Since there are 36 questions, I'll organize the answers into parts so they're easier to study and practice.

Part 1: Linked Lists, Stacks and Queues (Questions 1–10)

1\. Student Record Management – Singly Linked List

Algorithm

* Create a node containing register number, name, and marks.

* Insert new student records into the list.

* Delete a student using the register number.

* Search for a student using the register number.

* Display the records.

Python program

```python
class Node:
    def __init__(self, reg, name, marks):
        self.reg = reg
        self.name = name
        self.marks = marks
        self.next = None

head = None

def insert(reg, name, marks):
    global head
    new = Node(reg, name, marks)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
```

```
            temp = temp.next
        temp.next = new

def search(reg):
    temp = head
    while temp:
        if temp.reg == reg:
            print(temp.name, temp.marks)
            return
        temp = temp.next
    print("Student not found")

def delete(reg):
    global head
    temp = head
    prev = None
    while temp:
        if temp.reg == reg:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            print("Student deleted")
            return
        prev = temp
        temp = temp.next
    print("Student not found")

def display():
    temp = head
    while temp:
        print(temp.reg, temp.name, temp.marks)
        temp = temp.next

insert(101, "Arun", 85)
insert(102, "Bala", 90)

display()
search(102)
delete(101)
display()
```

Output

```text
101 Arun 85
102 Bala 90
Bala 90
Student deleted
102 Bala 90
```

2\. Music Playlist Management – Doubly Linked List

Algorithm

* Create nodes containing song names.

* Connect each node to the previous and next nodes.

* Insert and delete songs.

* Search for a song.

* Display songs in forward and backward directions.

Python program

```python
class Node:
    def __init__(self, song):
        self.song = song
        self.prev = None
        self.next = None

head = None

def insert(song):
    global head
    new = Node(song)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
            temp = temp.next
        temp.next = new
```

```
        new.prev = temp

def display_forward():
    temp = head
    while temp:
        print(temp.song, end=" ")
        temp = temp.next
    print()

def display_backward():
    temp = head
    if temp is None:
        return
    while temp.next:
        temp = temp.next
    while temp:
        print(temp.song, end=" ")
        temp = temp.prev
    print()

def search(song):
    temp = head
    while temp:
        if temp.song == song:
            print("Song found")
            return
        temp = temp.next
    print("Song not found")

def delete(song):
    global head
    temp = head
    while temp:
        if temp.song == song:
            if temp.prev:
                temp.prev.next = temp.next
            else:
                head = temp.next
            if temp.next:
                temp.next.prev = temp.prev
            print("Song deleted")
            return
        temp = temp.next
    print("Song not found")
```

```
insert("Song1")
insert("Song2")
insert("Song3")

print("Forward:")
display_forward()

print("Backward:")
display_backward()

search("Song2")
delete("Song2")
display_forward()
```

Output

```text
Forward:
Song1 Song2 Song3
Backward:
Song3 Song2 Song1
Song found
Song deleted
Song1 Song3
```

3\. Hospital Patient Management – Singly Linked List

Algorithm

* Create a node for each patient containing the patient ID and name.

* Add new patients to the waiting list.

* Remove patients after consultation.

* Search for a patient using the ID.

* Display the waiting list.

Python program

```python
class Node:
    def __init__(self, pid, name):
        self.pid = pid
        self.name = name
        self.next = None

head = None

def add(pid, name):
    global head
    new = Node(pid, name)
    if head is None:
        head = new
    else:
        temp = head
        while temp.next:
            temp = temp.next
        temp.next = new

def search(pid):
    temp = head
    while temp:
        if temp.pid == pid:
            print("Patient found:", temp.name)
            return
        temp = temp.next
    print("Patient not found")

def remove(pid):
    global head
    temp = head
    prev = None
    while temp:
        if temp.pid == pid:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            print("Patient removed")
            return
        prev = temp
        temp = temp.next
    print("Patient not found")
```

```
def display():
    temp = head
    while temp:
        print(temp.pid, temp.name)
        temp = temp.next

add(1, "Arun")
add(2, "Bala")
display()
search(2)
remove(1)
display()
```

Output

```text
1 Arun
2 Bala
Patient found: Bala
Patient removed
2 Bala
```

4\. Browser History Management – Doubly Linked List

Algorithm

* Store each visited website in a doubly linked list.

* Use previous and next links to navigate backward and forward.

* Delete a selected website from the history.

* Display the browsing history.

Python program

```python
class Node:
    def __init__(self, url):
        self.url = url
        self.prev = None
```

```
        self.next = None

head = None
current = None

def visit(url):
    global head, current
    new = Node(url)

    if head is None:
        head = new
    else:
        current.next = new
        new.prev = current

    current = new
    print("Visited:", url)

def backward():
    global current
    if current and current.prev:
        current = current.prev
        print("Current page:", current.url)
    else:
        print("No previous page")

def forward():
    global current
    if current and current.next:
        current = current.next
        print("Current page:", current.url)
    else:
        print("No next page")

def delete(url):
    global head, current
    temp = head

    while temp:
        if temp.url == url:
            if temp.prev:
                temp.prev.next = temp.next
            else:
                head = temp.next
```

```
            if temp.next:
                temp.next.prev = temp.prev

            if current == temp:
                current = temp.prev if temp.prev else temp.next

            print("History deleted")
            return
        temp = temp.next

    print("Website not found")

def display():
    temp = head
    while temp:
        print(temp.url)
        temp = temp.next

visit("Google")
visit("YouTube")
visit("Wikipedia")
backward()
forward()
delete("YouTube")
display()
```

Output

```text
Visited: Google
Visited: YouTube
Visited: Wikipedia
Current page: YouTube
Current page: Wikipedia
History deleted
Google
Wikipedia
```

5\. Calculator Application – Stack

Algorithm

* Read a postfix expression.

* Push operands onto a stack.

* When an operator appears, pop two operands.

* Perform the operation and push the result.

* Display the final result.

Python program

```python
def evaluate(expression):
    stack = []

    for item in expression.split():
        if item.isdigit():
            stack.append(int(item))
        else:
            b = stack.pop()
            a = stack.pop()

            if item == "+":
                stack.append(a + b)
            elif item == "-":
                stack.append(a - b)
            elif item == "*":
                stack.append(a * b)
            elif item == "/":
                stack.append(a / b)

    return stack.pop()

exp = input("Enter postfix expression: ")
print("Result:", evaluate(exp))
```

Sample output

```text
Enter postfix expression: 5 3 + 2 *
Result: 16
```

```

### 6\. Undo Operation in a Text Editor – Stack

Algorithm

* Store each editing action in a stack.

* Push every new action onto the stack.

* To undo, pop the most recent action.

* Display the action that was undone.

Python program

```python
stack = []

def perform(action):
    stack.append(action)
    print("Action performed:", action)

def undo():
    if stack:
        print("Undo:", stack.pop())
    else:
        print("Nothing to undo")

perform("Type Hello")
perform("Type Python")
perform("Delete text")

undo()
undo()
```

Output

```text
Action performed: Type Hello
Action performed: Type Python
Action performed: Delete text
Undo: Delete text
```
```

Undo: Type Python
```

### 7\. Printer Job Scheduling – Linear Queue

Algorithm

* Create an empty queue.

* Add print jobs at the rear.

* Process jobs from the front using FIFO order.

* Display the pending jobs.

Python program

```python
queue = []

def add_job(job):
    queue.append(job)
    print("Job added:", job)

def process_job():
    if queue:
        print("Printing:", queue.pop(0))
    else:
        print("Queue is empty")

def display():
    print("Pending jobs:", queue)

add_job("Document1")
add_job("Document2")
add_job("Document3")

display()
process_job()
display()
```

Output
```

```text
Job added: Document1
Job added: Document2
Job added: Document3
Pending jobs: ['Document1', 'Document2', 'Document3']
Printing: Document1
Pending jobs: ['Document2', 'Document3']
```

8\. Customer Service Management – Circular Queue

Algorithm

* Create a fixed-size queue.

* Insert customers using the rear position.

* Serve customers from the front.

* Use modulo arithmetic to reuse empty positions.

Python program

```python
size = 3
queue = [None] * size
front = -1
rear = -1

def add(customer):
    global front, rear

    if (rear + 1) % size == front:
        print("Queue is full")
        return

    if front == -1:
        front = 0

    rear = (rear + 1) % size
    queue[rear] = customer
    print("Added:", customer)

def serve():
```

```
    global front, rear

    if front == -1:
        print("Queue is empty")
        return

    print("Serving:", queue[front])
    queue[front] = None

    if front == rear:
        front = rear = -1
    else:
        front = (front + 1) % size

def display():
    if front == -1:
        print("Queue is empty")
        return

    i = front
    while True:
        print(queue[i], end=" ")
        if i == rear:
            break
        i = (i + 1) % size
    print()

add("Arun")
add("Bala")
add("Cathy")
serve()
add("David")
display()
```

Output

```text
Added: Arun
Added: Bala
Added: Cathy
Serving: Arun
Added: David
Bala Cathy David
```

```

### 9\. Expression Validation – Stack

Algorithm

* Read the expression character by character.

* Push opening brackets `(`, `[` and `{` onto the stack.

* For each closing bracket, check the top of the stack.

* If the brackets match, pop the opening bracket.

* If all brackets match and the stack is empty, the expression is balanced.

Python program

```python
def check(expression):
    stack = []
    pairs = {')': '(', ']': '[', '}': '{'}

    for ch in expression:
        if ch in "([{":
            stack.append(ch)

        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False

    return len(stack) == 0

exp = input("Enter expression: ")

if check(exp):
    print("Balanced expression")
else:
    print("Not balanced")
```

Sample output

```text

Enter expression: {[a+b]*(c+d)}
Balanced expression
```

10\. Postfix Expression Evaluation

Question: Evaluate the postfix expression `8 3 2 * + 5 -` using stack operations.

Algorithm

* Push `8` onto the stack.

* Push `3` and `2`.

* Apply multiplication: .

* Apply addition: .

* Subtract `5`: .

* Display the result.

Python program

```python
stack = []
expression = "8 3 2 * + 5 -"

for item in expression.split():
    if item.isdigit():
        stack.append(int(item))
    else:
        b = stack.pop()
        a = stack.pop()

        if item == "*":
            stack.append(a * b)
        elif item == "+":
            stack.append(a + b)
        elif item == "-":
            stack.append(a - b)

print("Result:", stack.pop())
```
```

Output

```text
Result: 9
```

Part 2: Recursion, Trees and Priority Queues (Questions 11–20)

11\. Product Arrangement Calculation – Factorial

Algorithm

* Read a number.

* Calculate factorial using recursion.

* Calculate factorial using iteration.

* Display both results.

Python program

```python
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

n = int(input("Enter a number: "))

print("Recursive:", factorial_recursive(n))
print("Iterative:", factorial_iterative(n))
```

Output

```text
```

```
Enter a number: 5
Recursive: 120
Iterative: 120
```

12\. Population Growth Prediction – Fibonacci Series

Algorithm

* Read the number of terms.

* Generate Fibonacci numbers using recursion.

* Generate Fibonacci numbers using iteration.

* Display both series.

Python program

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter number of terms: "))

print("Recursive:")
for i in range(n):
    print(fib(i), end=" ")

print("\nIterative:")
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output

```text
Enter number of terms: 6
Recursive:
0 1 1 2 3 5
```

Iterative:
0 1 1 2 3 5
```

### 13\. Library Book Search – Binary Search

Algorithm

* Store book IDs in sorted order.

* Find the middle element.

* Compare it with the required ID.

* Search the left or right half.

* Repeat until the ID is found or the search range becomes empty.

Python program

```python
def binary_iterative(a, key):
    low, high = 0, len(a) - 1

    while low <= high:
        mid = (low + high) // 2

        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

def binary_recursive(a, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if a[mid] == key:
        return mid
```
```

```
    elif a[mid] < key:
        return binary_recursive(a, mid + 1, high, key)
    else:
        return binary_recursive(a, low, mid - 1, key)

a = [10, 20, 30, 40, 50]
key = int(input("Enter book ID: "))

print("Iterative position:", binary_iterative(a, key))
print("Recursive position:",
      binary_recursive(a, 0, len(a) - 1, key))
```

Output

```text
Enter book ID: 30
Iterative position: 2
Recursive position: 2
```

Note: Positions start from index 0.

14\. Examination Seating Arrangement – Factorial

This question uses the same factorial method as Question 11.

Python program

```python
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def factorial_loop(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

n = int(input("Enter number of students: "))

print("Recursion:", factorial(n))
```

```
print("Iteration:", factorial_loop(n))
```

Output

```text
Enter number of students: 4
Recursion: 24
Iteration: 24
```

15\. Rabbit Population – Fibonacci Series

This question uses the same Fibonacci methods as Question 12.

Python program

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

n = int(input("Enter number of terms: "))

print("Using recursion:")
for i in range(n):
    print(fib(i), end=" ")

print("\nUsing iteration:")
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output

```text
Enter number of terms: 5
Using recursion:
0 1 1 2 3
Using iteration:
0 1 1 2 3
```

```

### 16\. Employee ID Search – Binary Search

Python program

```python
def iterative_search(a, key):
    low, high = 0, len(a) - 1

    while low <= high:
        mid = (low + high) // 2

        if a[mid] == key:
            return mid
        elif a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

def recursive_search(a, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if a[mid] == key:
        return mid
    elif a[mid] < key:
        return recursive_search(a, mid + 1, high, key)
    else:
        return recursive_search(a, low, mid - 1, key)

ids = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID: "))

print("Iterative:", iterative_search(ids, key))
print("Recursive:",
      recursive_search(ids, 0, len(ids) - 1, key))
```

Output
```

```text
Enter employee ID: 104
Iterative: 3
Recursive: 3
```

17\. Employee Record Management – Binary Search Tree

Algorithm

* Insert employee IDs into a binary search tree.

* Place smaller IDs on the left and larger IDs on the right.

* Search for a given ID.

* Display inorder, preorder and postorder traversals.

Python program

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
```

```
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

def search(root, key):
    if root is None or root.data == key:
        return root

    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)

root = None
for x in [50, 30, 70, 20, 40]:
    root = insert(root, x)

print("Inorder:")
inorder(root)

print("\nPreorder:")
preorder(root)

print("\nPostorder:")
postorder(root)

print("\nSearch 40:", "Found" if search(root, 40) else "Not found")
```

Output

```text
Inorder:
20 30 40 50 70
Preorder:
50 30 20 40 70
```

Postorder:
20 40 30 70 50
Search 40: Found
```

### 18\. Student Mark Management – Binary Search Tree

Python program

```python
class Node:
    def __init__(self, mark):
        self.mark = mark
        self.left = None
        self.right = None

def insert(root, mark):
    if root is None:
        return Node(mark)

    if mark < root.mark:
        root.left = insert(root.left, mark)
    elif mark > root.mark:
        root.right = insert(root.right, mark)

    return root

def inorder(root):
    if root:
        inorder(root.left)
        print(root.mark, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.mark, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.mark, end=" ")
```
```

```
root = None
for mark in [80, 60, 90, 50, 70]:
    root = insert(root, mark)

print("Inorder:")
inorder(root)
print("\nPreorder:")
preorder(root)
print("\nPostorder:")
postorder(root)
```

Output

```text
Inorder:
50 60 70 80 90
Preorder:
80 60 50 70 90
Postorder:
50 70 60 90 80
```

19\. Arithmetic Expression Evaluation – Expression Tree

Algorithm

* Read a postfix expression.

* Create a node for each operand.

* When an operator appears, pop two nodes and make them its children.

* Push the new operator node onto the stack.

* Evaluate the completed expression tree.

Python program

```python
class Node:
    def __init__(self, value):
        self.value = value
```

```
        self.left = None
        self.right = None

def build_tree(exp):
    stack = []

    for item in exp.split():
        node = Node(item)
```